

# Búsqueda informada y heurísticas

De costo uniforme a  $A^*$ : diseño, garantías y evaluación

Curso de Inteligencia Artificial  
Semana 3 · Clase 3

# Propósito de la clase

Incorporar conocimiento del problema para orientar la búsqueda sin confundir rapidez con garantía.

## Al finalizar, podremos

- ▶ definir y auditar una función heurística;
- ▶ ejecutar búsqueda voraz y  $A^*$ ;
- ▶ distinguir admisibilidad de consistencia;
- ▶ explicar cuándo  $A^*$  requiere reaperturas;
- ▶ diseñar una comparación reproducible entre UCS y  $A^*$ .

# Punto de partida

La clase anterior dejó establecidos:

- ▶ el problema  $P = (S, A, T, s_0, G, c)$ ;
- ▶ nodo, frontera y conjunto explorado;
- ▶ control de ciclos y mejores costos;
- ▶ búsqueda en anchura (*Breadth-First Search*, BFS), búsqueda en profundidad (*Depth-First Search*, DFS) y búsqueda de costo uniforme (*Uniform-Cost Search*, UCS);
- ▶ garantías condicionadas por representación y costos.

## Continuidad

UCS ordena por costo recorrido; ahora agregaremos una estimación del costo restante.

# Pregunta de apertura

## Pregunta clave

Si dos caminos cuestan lo mismo hasta ahora, ¿cómo decidir cuál explorar primero?

- ▶ ¿Qué conocimiento adicional está permitido?
- ▶ ¿En qué unidades debe expresarse?
- ▶ ¿Cómo sabemos si orienta sin engañar?
- ▶ ¿Qué garantía queremos conservar?

# El esquema de búsqueda no cambia

```
insertar nodo inicial en frontera
mientras frontera no esté vacía:
    extraer el nodo con menor valor de prioridad
    descartar entradas obsoletas
    si es objetivo: devolver camino
    relajar cada transición legal
devolver fracaso
```

La estrategia cambia al definir la prioridad de la frontera.

# Una familia, distintas prioridades

| Estrategia | Prioridad               | Pregunta que privilegia                |
|------------|-------------------------|----------------------------------------|
| BFS        | profundidad             | ¿Cuántos pasos llevo?                  |
| DFS        | LIFO; mayor profundidad | ¿Qué rama continúo primero?            |
| UCS        | $g(n)$                  | ¿Cuánto costó llegar?                  |
| Voraz      | $h(n)$                  | ¿Qué parece más cerca?                 |
| A*         | $g(n) + h(n)$           | ¿Qué solución total parece más barata? |

## Siglas

**BFS:** *Breadth-First Search* (búsqueda en anchura).

**DFS:** *Depth-First Search* (búsqueda en profundidad).

**UCS:** *Uniform-Cost Search* (búsqueda de costo uniforme).

# Búsqueda no informada e informada

Ambas parten de  $P = (S, A, T, s_0, G, c)$ . La diferencia es la información usada para ordenar la frontera.

## No informada: sin $h$

- ▶ Usa la definición del problema y el costo recorrido.
- ▶ No estima el costo que falta.
- ▶ Incluye BFS, DFS, limitada, iterativa y UCS.
- ▶ Prioriza por profundidad, orden o  $g(n)$ .

UCS pertenece aquí:  $g(n)$  es costo exacto del pasado, no una estimación del futuro.

## Informada: incorpora $h$

- ▶ Añade una estimación dirigida al objetivo.
- ▶ Voraz prioriza  $h(n)$ .
- ▶ A\* prioriza  $g(n) + h(n)$ .
- ▶ La garantía depende de la estrategia y de  $h$ .

Voraz es informada, pero no es óptima en general.

No informada no significa aleatoria; informada no significa conocer el futuro ni garantizar optimalidad.

# Punto de control: costo uniforme

UCS extrae el nodo con menor  $g(n)$ .

- ▶ Usa una cola de prioridad.
- ▶ Conserva el mejor costo conocido por estado.
- ▶ Prueba el objetivo al extraer, no al generar.
- ▶ Descarta entradas obsoletas.
- ▶ Es óptima bajo los supuestos declarados.

## Límite

Puede expandir muchas alternativas baratas que se alejan del objetivo.



# ¿Qué añade la búsqueda informada?

Añade conocimiento para ordenar la exploración:

$$h(n) \approx \text{costo mínimo que falta desde } n \text{ hasta un objetivo}$$

- ▶ No altera  $g(n)$ .
- ▶ No crea conexiones inexistentes.
- ▶ No corrige estados insuficientes.
- ▶ No reemplaza restricciones.

Una heurística guía la búsqueda dentro del problema ya formulado.

# Función heurística

Una heurística es una función:

$$h : S \rightarrow \mathbb{R}$$

- ▶  $h(n)$ : estimación del costo restante;
- ▶  $h^*(n)$ : costo óptimo restante verdadero;
- ▶  $h(g) = 0$  para todo estado objetivo  $g$ .

## Convención

La heurística se evalúa sobre estados, aunque se escriba  $h(n)$  por conveniencia.

# Tres cantidades diferentes

| Cantidad | Significado                   | Disponibilidad              |
|----------|-------------------------------|-----------------------------|
| $g(n)$   | costo acumulado desde $s_0$   | conocido al generar         |
| $h(n)$   | estimación del costo restante | calculado por la heurística |
| $f(n)$   | prioridad de la estrategia    | depende del algoritmo       |

En A\*:

$$f(n) = g(n) + h(n)$$

$f$  estima el costo total de una solución que pasa por  $n$ .

# Cómo interpretar $g(n)$ , $h(n)$ y $f(n)$

Para  $C$ , alcanzado mediante  $S \rightarrow A \rightarrow C$ :

| Función | Cálculo     | Interpretación                             |
|---------|-------------|--------------------------------------------|
| $g(C)$  | $2 + 2 = 4$ | costo exacto del camino recorrido          |
| $h(C)$  | 3           | estimación del costo restante hasta $G$    |
| $f(C)$  | $4 + 3 = 7$ | estimación del costo total pasando por $C$ |

- ▶  $g$  depende del camino;  $h$ , del estado y del objetivo.
- ▶  $f$  evalúa prioridad: no es un costo ya pagado.
- ▶ UCS prioriza  $g$ ; voraz,  $h$ ;  $A^*$ ,  $f = g + h$ .

**$g$  resume el pasado,  $h$  estima el futuro y  $f$  combina ambos.**

# La unidad debe coincidir

Si  $g$  está en minutos,  $h$  también debe estar en minutos.

| Costo objetivo    | Heurística candidata                |
|-------------------|-------------------------------------|
| distancia         | separación geométrica inferior      |
| tiempo            | distancia / velocidad máxima válida |
| cantidad de pasos | mínimo de pasos pendientes          |

## Inválido

Sumar kilómetros y minutos sin una conversión justificada.

# Dos extremos útiles

## Heurística nula

$$h(n) = 0$$

A\* se convierte en UCS. Es segura, pero no aporta orientación.

## Heurística perfecta

$$h(n) = h^*(n)$$

Orienta idealmente, pero calcularla suele equivaler a resolver el problema.

**Buscamos una aproximación barata, informativa y justificable.**

# Heurística admisible

Una heurística es admisible si nunca sobreestima:

$$0 \leq h(n) \leq h^*(n)$$

- ▶ Es una cota inferior, no una predicción promedio.
- ▶ Debe cumplirse para todos los estados relevantes.
- ▶ Una sola sobreestimación rompe la garantía general.

## Alcance

Admisibilidad se refiere al costo óptimo restante global.

# Búsqueda voraz por el mejor primero

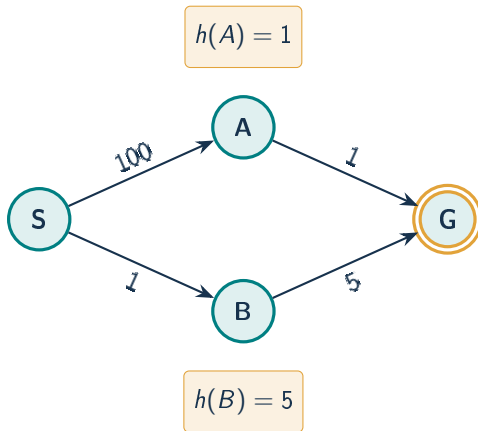
La búsqueda voraz ordena la frontera solo por:

$$f(n) = h(n)$$

- ▶ Elige el estado que parece más cercano al objetivo.
- ▶ Ignora cuánto costó llegar hasta allí.
- ▶ Puede encontrar rápidamente una solución plausible.
- ▶ No garantiza el menor costo.



# Contraejemplo para la búsqueda voraz



Voraz elige A y devuelve costo 101; existe  $S \rightarrow B \rightarrow G$  de costo 6.

# ¿Por qué falla la voraz?

| Nodo | $g$ | $h$ | prioridad voraz |
|------|-----|-----|-----------------|
| A    | 100 | 1   | 1               |
| B    | 1   | 5   | 5               |

## Lectura

La heurística describe costo restante, no costo total.

Al ignorar  $g$ , una promesa cercana puede ocultar un pasado extremadamente caro.

# Alcance de la búsqueda voraz

- ▶ En un grafo finito con control de repetidos, termina.
- ▶ En espacios infinitos no es completa en general.
- ▶ No es óptima, incluso si  $h$  es admisible.
- ▶ Debe evaluarse tanto velocidad como calidad de solución.

## Uso razonable

Obtener una solución rápida cuando la optimalidad no es obligatoria.

# Algoritmo A\*

A\* equilibra lo recorrido y lo que falta:

$$f(n) = g(n) + h(n)$$

- ▶  $g(n)$  aporta evidencia del camino ya recorrido.
- ▶  $h(n)$  orienta hacia regiones prometedoras.
- ▶ Se extrae el nodo con menor  $f$ .

## Caso límite

Con  $h = 0$ , A\* reproduce el comportamiento de UCS.

# Qué debe guardar A\*

Cada nodo conserva:

$$n = (\text{estado}, \text{padre}, \text{acción}, g, h, f)$$

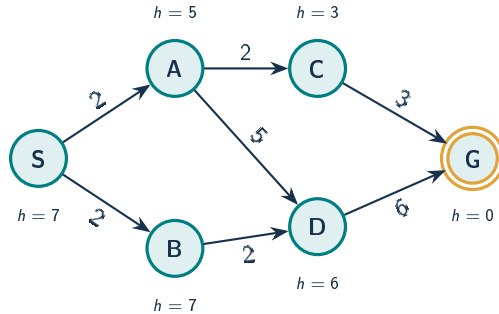
La implementación también necesita:

- ▶ cola de prioridad por  $f$ ;
- ▶ `mejor_g[estado]`;
- ▶ regla de desempate estable;
- ▶ padres para reconstruir el camino;
- ▶ política explícita de reapertura.

# Pseudocódigo de A\*

```
frontera <- prioridad por f
mejor_g[inicial] <- 0
insertar inicial con f = h(inicial)
mientras haya frontera:
    n <- extraer mínimo
    si n.g es obsoleto: continuar
    si n es objetivo: devolver camino
    para cada sucesor:
        nuevo_g <- n.g + costo
        si nuevo_g mejora mejor_g:
            registrar e insertar con f = nuevo_g + h
devolver fracaso
```

# Grafo didáctico para A\*



Aristas dirigidas · costos no negativos · desempate por orden de inserción.

# A\*: inicialización

| Nodo | $g$ | $h$ | $f$ |
|------|-----|-----|-----|
| S    | 0   | 7   | 7   |

## Primera extracción

Se extrae  $S$  porque es el único nodo disponible.

Regla de desempate del ejemplo: **orden de inserción**.



## A\*: después de expandir S

| Frontera | $g$ | $h$ | $f$ |
|----------|-----|-----|-----|
| A        | 2   | 5   | 7   |
| B        | 2   | 7   | 9   |

### Decisión

A\* extrae A. Ambos caminos recorridos cuestan lo mismo; la heurística rompe la igualdad.

## A\*: de A al objetivo

| Frontera tras expandir A | $g$ | $h$ | $f$ |
|--------------------------|-----|-----|-----|
| C                        | 4   | 3   | 7   |
| B                        | 2   | 7   | 9   |
| D                        | 7   | 6   | 13  |

Al expandir  $C$  aparece  $G$  con  $g = f = 7$ . Al extraer  $G$ :

$$S \rightarrow A \rightarrow C \rightarrow G, \quad C^* = 7$$

# UCS y A\* sobre la misma instancia

| Resultado                     | UCS             | A*              |
|-------------------------------|-----------------|-----------------|
| Camino                        | $S - A - C - G$ | $S - A - C - G$ |
| Costo                         | 7               | 7               |
| Prioridad                     | $g$             | $g + h$         |
| Expandidos antes de extraer G | $S, A, B, C, D$ | $S, A, C$       |

La heurística reduce expansiones en este ejemplo; no reduce el costo óptimo.

# A\* no significa óptimo automáticamente

La garantía depende de:

- ▶ ramificación finita;
- ▶ costos no negativos y, para terminación,  $c \geq \varepsilon > 0$ ;
- ▶ objetivo comprobado al extraer;
- ▶ conservación del mejor  $g$ ;
- ▶ descarte de entradas obsoletas;
- ▶ reapertura cuando sea necesaria;
- ▶ propiedades demostradas de  $h$ ;
- ▶ unidades compatibles.

El nombre del algoritmo no sustituye estas condiciones.

# Ejemplo de heurística admisible

En el grafo didáctico podemos calcular el costo óptimo restante  $h^*(n)$ :

| Estado | $h(n)$ | $h^*(n)$ | Comprobación |
|--------|--------|----------|--------------|
| $S$    | 7      | 7        | $7 \leq 7$   |
| $A$    | 5      | 5        | $5 \leq 5$   |
| $B$    | 7      | 8        | $7 \leq 8$   |
| $C$    | 3      | 3        | $3 \leq 3$   |
| $D$    | 6      | 6        | $6 \leq 6$   |
| $G$    | 0      | 0        | $0 \leq 0$   |

La heurística puede subestimar y seguir siendo admisible; no necesita coincidir con  $h^*$ .

## Idea clave

Admisible significa no sobreestimar, no acertar exactamente.

# Intuición de optimalidad

Sea  $C^*$  el costo de la solución óptima. Para un nodo  $n$  sobre un camino óptimo, la admisibilidad implica:

$$f(n) = g(n) + h(n) \leq g(n) + h^*(n) = C^*$$

En una meta,  $h(G) = 0$ . Para una solución subóptima de costo  $C > C^*$ :

$$f(G) = g(G) = C > C^*$$

## Implicancia

Mientras la frontera conserve un nodo del camino óptimo con  $f(n) \leq C^*$ ,  $A^*$  lo extraerá antes que una meta subóptima con  $f(G) > C^*$ .

## Ejemplo

Si  $C^* = 7$  y aparece una meta de costo 9, un candidato óptimo con  $f \leq 7$  tiene prioridad sobre esa meta.

Esta conclusión requiere comprobar la meta al extraerla y respetar las demás condiciones de  $A^*$ .

# Heurística consistente

Para toda transición  $n \rightarrow n'$ :

$$h(n) \leq c(n, n') + h(n')$$

Es una desigualdad triangular. Entonces:

$$f(n') = g(n) + c(n, n') + h(n') \geq g(n) + h(n) = f(n)$$

## Consecuencia

Con  $h(G) = 0$  y  $h(n) \geq 0$ , consistencia implica la admisibilidad adoptada y los valores  $f$  no disminuyen.

# Auditoría local de consistencia

| Arista            | Comprobación   | Arista            | Comprobación   |
|-------------------|----------------|-------------------|----------------|
| $S \rightarrow A$ | $7 \leq 2 + 5$ | $S \rightarrow B$ | $7 \leq 2 + 7$ |
| $A \rightarrow C$ | $5 \leq 2 + 3$ | $A \rightarrow D$ | $5 \leq 5 + 6$ |
| $B \rightarrow D$ | $7 \leq 2 + 6$ | $C \rightarrow G$ | $3 \leq 3 + 0$ |
| $D \rightarrow G$ | $6 \leq 6 + 0$ |                   |                |

Cero violaciones demuestra consistencia **en este grafo revisado**, no en datos futuros.



# Admisible pero inconsistente

Admisibilidad no garantiza consistencia.

- ▶ Los valores  $f$  pueden disminuir a lo largo de un camino.
- ▶ Puede aparecer después un camino más barato hacia un estado cerrado.
- ▶ A\* debe permitir la **reapertura** cuando mejora  $mejor\_g$ .
- ▶ Cerrar estados para siempre puede producir una solución subóptima.

## Regla de implementación

No prohibir reaperturas sin haber demostrado consistencia.

# Cómo diseñar una heurística

| Principio             | Idea                                                    |
|-----------------------|---------------------------------------------------------|
| <b>Relajación</b>     | eliminar restricciones y resolver el problema más fácil |
| <b>Abstracción</b>    | buscar exactamente en un espacio reducido               |
| <b>Geometría</b>      | usar distancias y límites físicos                       |
| <b>Descomposición</b> | combinar subcostos sin doble conteo                     |
| <b>Patrones</b>       | precomputar costos de configuraciones abstractas        |

La procedencia de la cota debe quedar documentada.

# Combinar y evaluar heurísticas

Si  $h_1$  y  $h_2$  son admisibles:

- ▶  $\max(h_1, h_2)$  también es admisible y domina a ambas;
- ▶  $h_1 + h_2$  requiere costos aditivos sin superposición.

## Evaluar sobre las mismas instancias

- ▶ costo de solución;
- ▶ nodos generados y expandidos;
- ▶ frontera máxima y reaperturas;
- ▶ tiempo de búsqueda y costo de calcular  $h$ .

# Comparación de estrategias

| Estrategia | Prioridad               | Óptima          | Riesgo principal         |
|------------|-------------------------|-----------------|--------------------------|
| BFS        | profundidad             | costos iguales* | memoria                  |
| DFS        | LIFO; mayor profundidad | no              | ramas profundas y ciclos |
| UCS        | $g$                     | sí*             | expansión amplia         |
| Voraz      | $h$                     | no              | solución cara            |
| A*         | $g + h$                 | sí*             | memoria y supuestos      |

\* Bajo ramificación finita, costo mínimo positivo cuando corresponda, prueba correcta del objetivo, repetidos y propiedad heurística declarada.